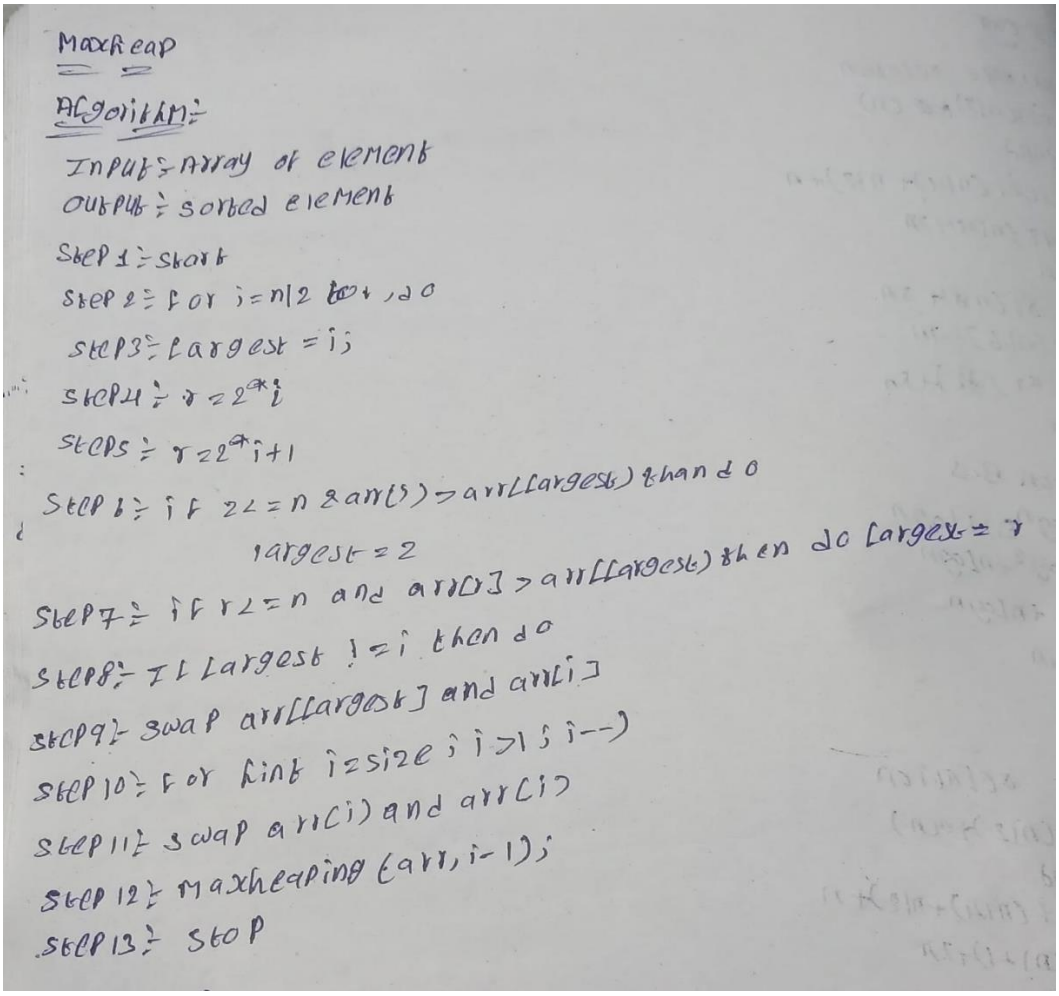
PRACTICAL – 3

AIM: Implementation of max-heap sort algorithm .

Max heap sort Algorithm :



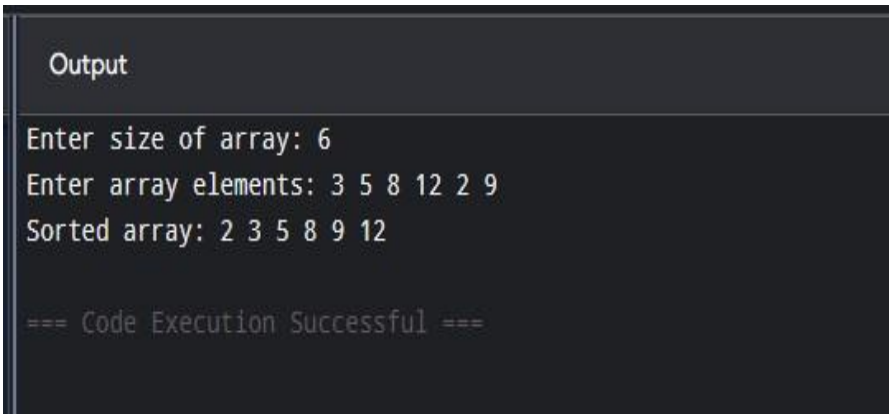
CODE:

```
#include <iostream> using
namespace std; void heapify(int
arr[], int n, int i)
```

```
{  int largest = i;  int left = 2 * i + 1;
int right = 2 * i + 2;  if (left < n &&
arr[left] > arr[largest])  largest = left;

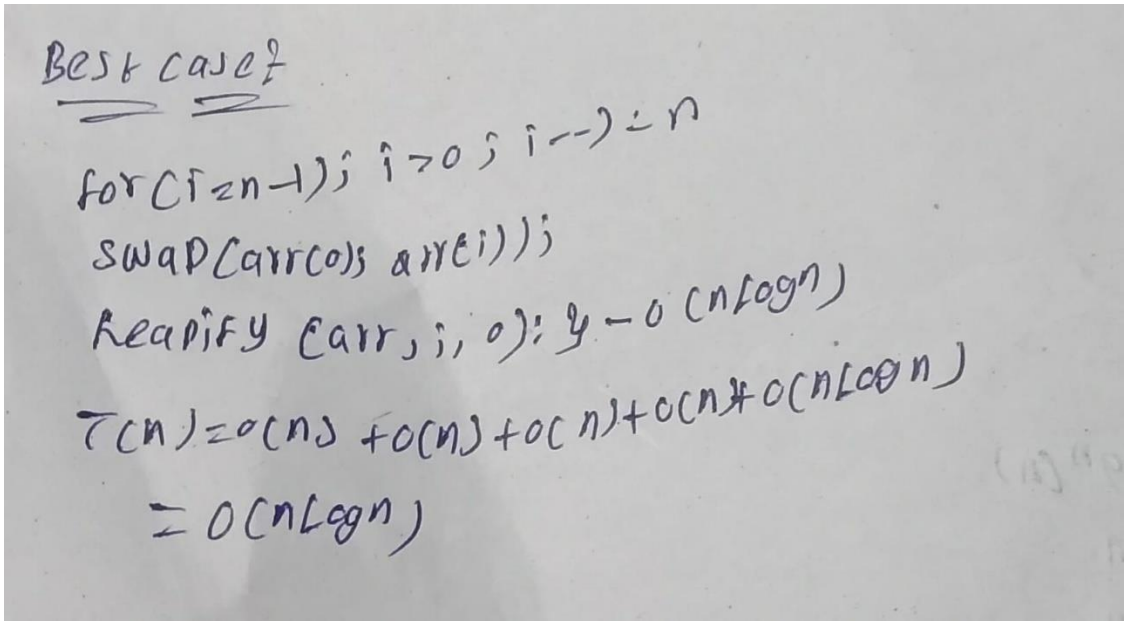

if (right < n && arr[right] > arr[largest])
largest = right;  if (largest != i)
 {  swap(arr[i],
arr[largest]);  heapify(arr,
n, largest);
 } } void heapSort(int arr[],
int n)
{
 // Build Max Heap
 for (int i = n / 2 - 1; i >= 0; i--)
heapify(arr, n, i);  // Heap Sort
 for (int i = n - 1; i > 0; i--)
 {  swap(arr[0],
arr[i]);  heapify(arr,
i, 0);
 } } int main() {  int n;  int
arr[100];  cout << "Enter size of
array: ";  cin >> n; cout <<
"Enter array elements: ";  for (int
i = 0; i < n; i++)  cin >> arr[i];
heapSort(arr, n);  cout <<
"Sorted array: ";  for (int i = 0; i
< n; i++)  cout << arr[i] << " ";
return 0;
}
```

OUTPUT:

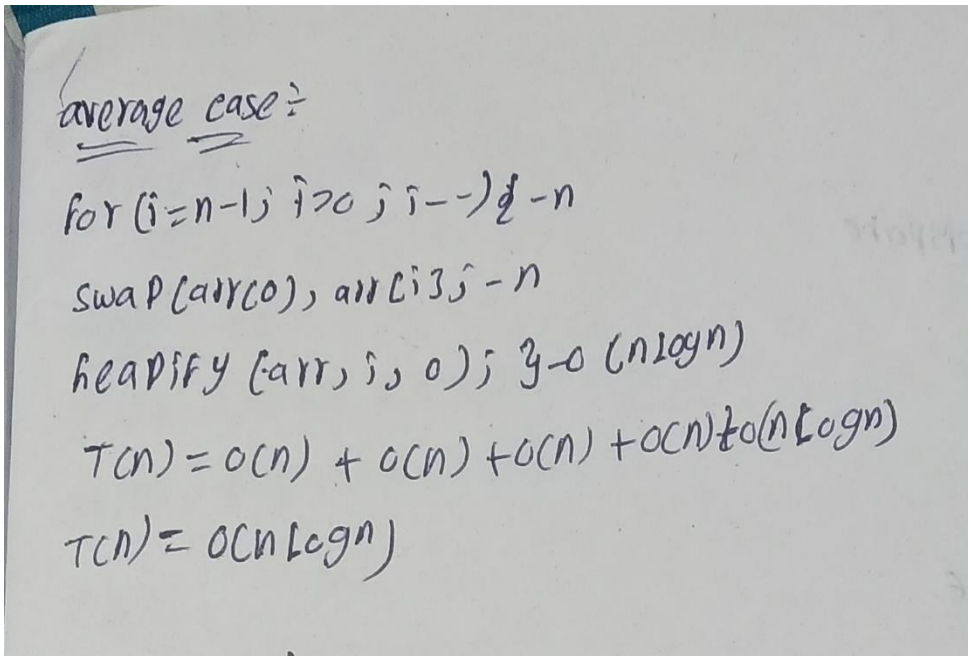


TIME COMPLEXITY:

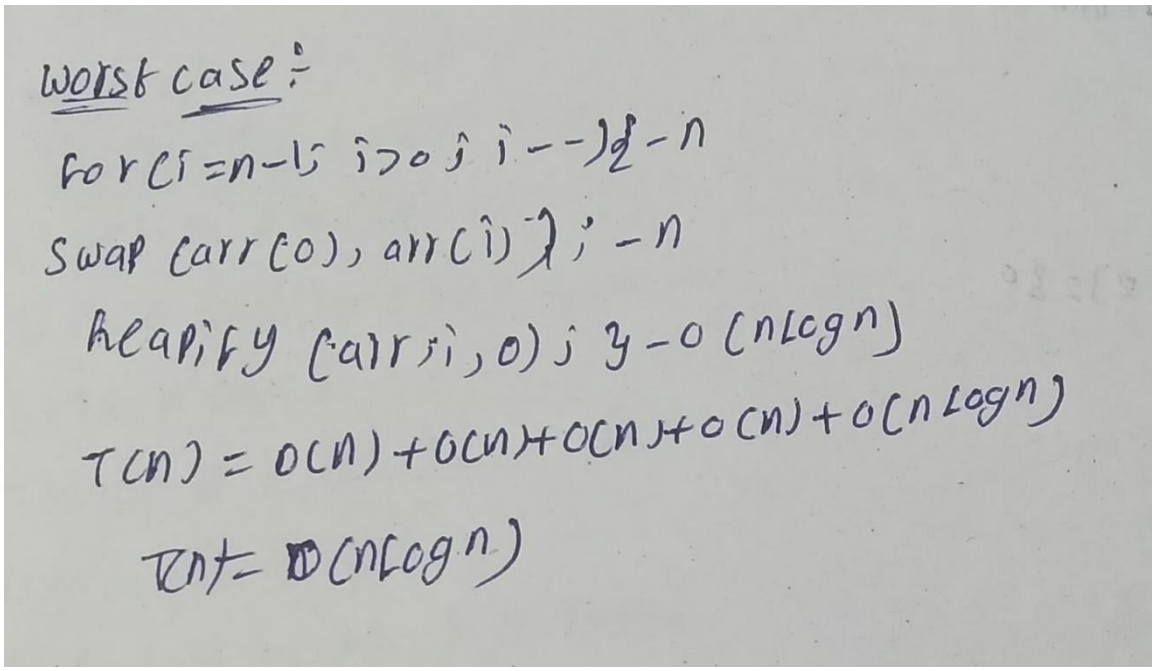
Best Case:



Average Case:



Worst Case:



CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented and tested. It sorts the elements by creating a max-heap and repeatedly placing the largest element in its correct position. It is efficient for large amounts of data and has a time complexity of O(n log n).